



Université Abou Bakr Belkaid
Faculté des sciences
Département d'informatique

COURS GÉNIE LOGICIEL

NIVEAU : L3 INFORMATIQUE/3 EME INGÉNIEUR

Chargée de cours : S Meziane Tani

E –mail : s.mezianetani13@gmail.com

Novembre 2025

Rappel

Spécification

- **Diagrammes de cas d'utilisation** : besoins des utilisateurs
- **Diagrammes de séquence** : scénarios d'interactions entre les utilisateurs et le logiciel, vu de l'extérieur
- **Diagrammes d'activité** : enchaînement d'actions représentant un comportement du logiciel

Conception

- **Diagrammes de classes** : structure interne du logiciel
- **Diagrammes d'objet** : état interne du logiciel à un instant donné
- **Diagrammes états-transitions** : évolution de l'état d'un objet
- **Diagrammes de séquence** : scénarios d'interactions avec les utilisateurs ou au sein du logiciel
- **Diagrammes de composants** : composants physiques du logiciel
- **Diagrammes de déploiement** : organisation matérielle du log

Notation de conception

- Pas de notation standard unique
- Modélisation du système à réaliser

UML

- Standard universel pour une conception **orientée objet**

Conception orientée objet

- Représentation du système comme **un ensemble d'objets interagissant**

–

- Conception fondée sur la notion de l'objet
- Inspirée des objets du monde réel
- Abstraction et décomposition du système en objets

- **Intérêts**

- Indépendance entre le modèle et les fonctions demandé
- Capacité d'adaptation et d'évolution => Maintenabilité

- **UML**

- Standard universel pour une conception **orientée objet**

Concepts orientés objet

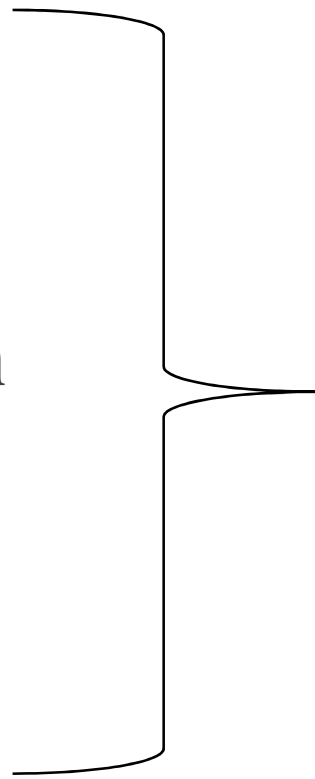
□ **Classe**

□ **Généralisation / spécialisation**

□ **Association**

□ **Agrégation / composition**

Diagramme de classe



Chapitre 5:

Diagramme de classes

Diagramme de classes

–

Représentation de la structure interne et logique du système

- Classes
- Relations entre classes

Classe

—

➤ **Type d'objets ayant les mêmes**

- Caractéristiques $\Leftrightarrow$ Attributs (informations, propriétés, etc.)
- Comportement $\Leftrightarrow$ Opérations (méthodes, messages, etc.)

➤ **Chaque objet est une instance d'une classe.**

Objets

Objet = Identité + Etat + Comportement

—

➤ Une identité

- deux objets différents ont des identités différentes
- on peut désigner l'objet (y faire référence)

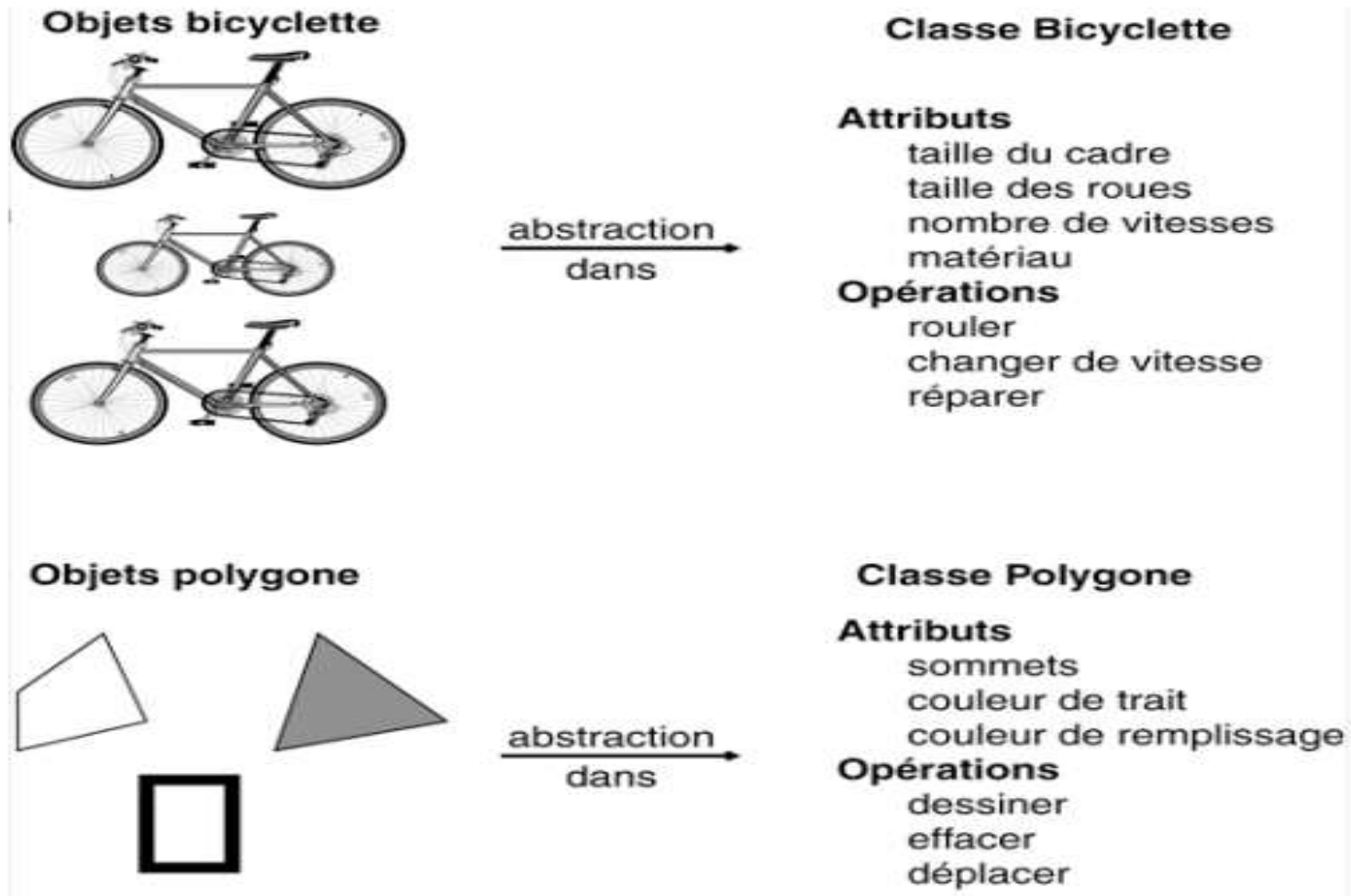
➤ Un état (attributs)

- ensemble de propriétés/caractéristiques définies par des valeurs
- permet de le personnaliser/distinguer des autres objets
- peut évoluer dans le temps

➤ Un comportement (méthodes)

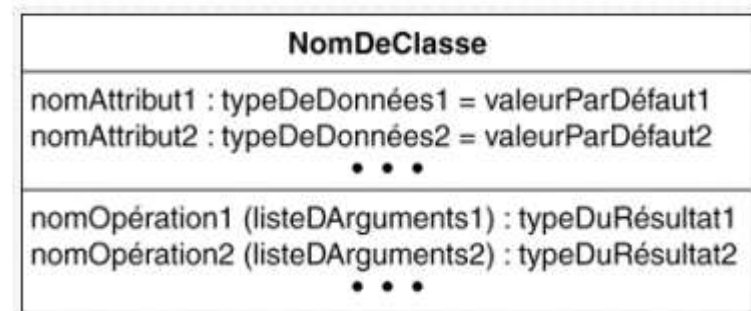
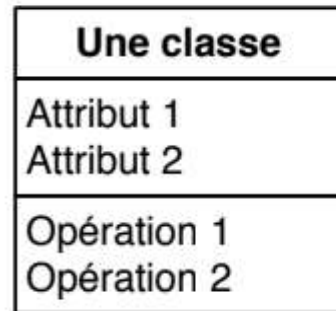
- ensemble des traitements que peut accomplir un objet (ou que l'on peut lui faire accomplir)

Exemple(Classes et instances)

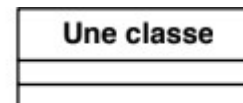
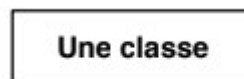


Représentation d'une classe

- Rectangle à 3 compartiments
 - nom (singulier, majuscule, Significatif)
 - attributs
 - opérations

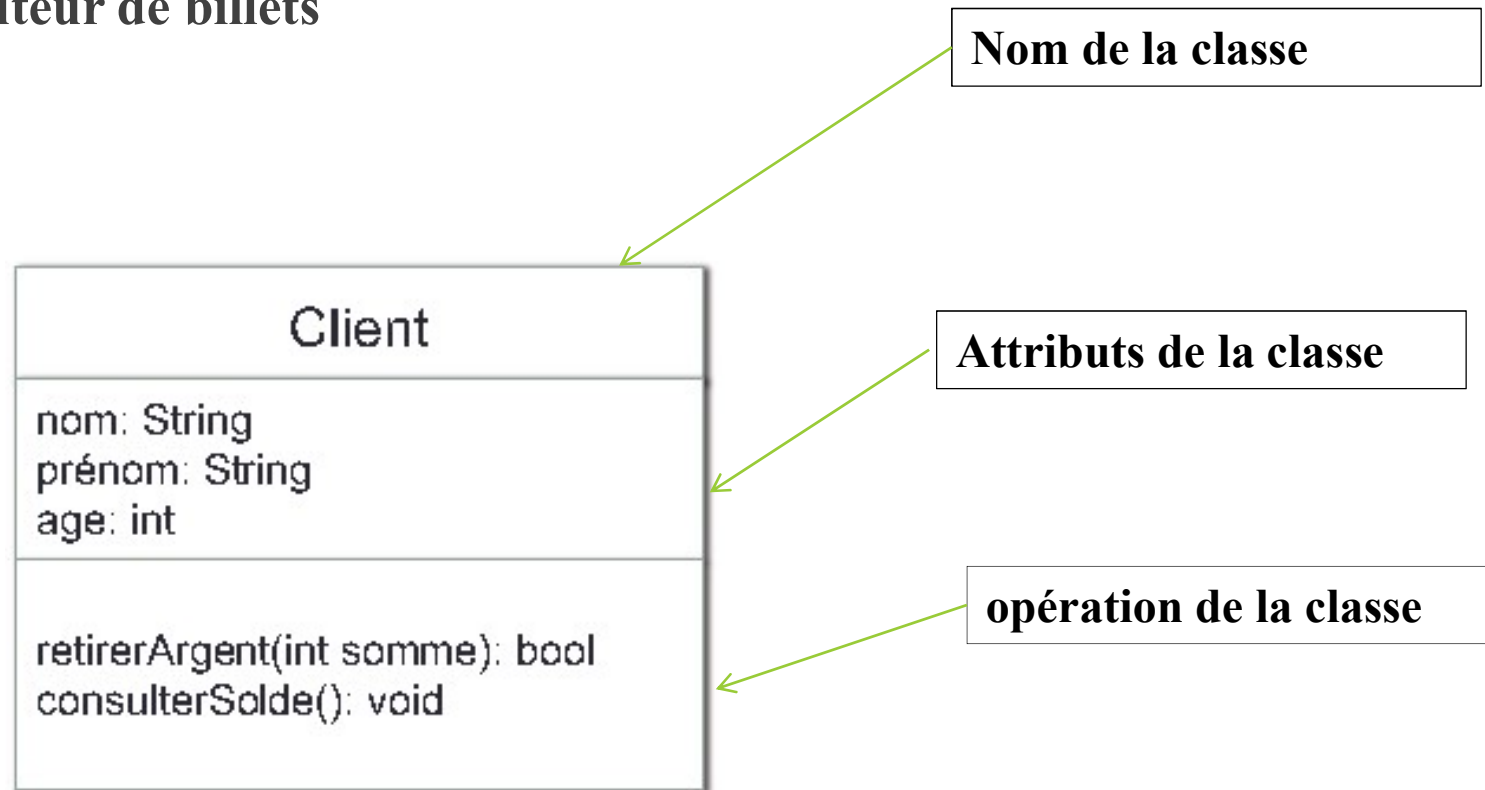


Plus ou moins de détails en fonction des besoins

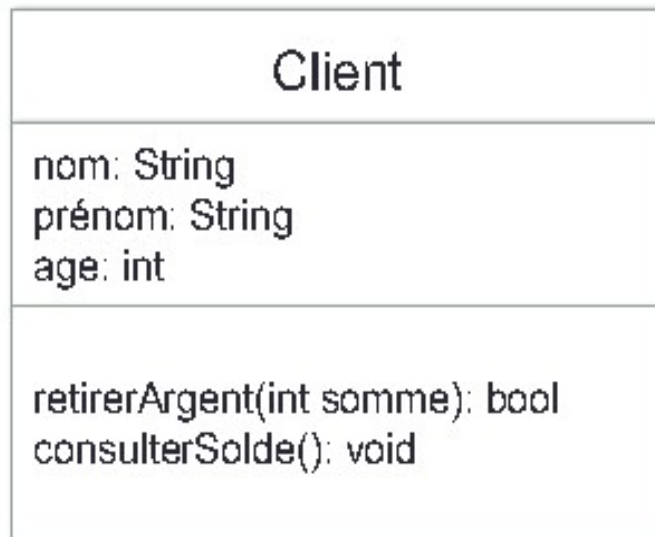


Exemple de classe

Distributeur de billets



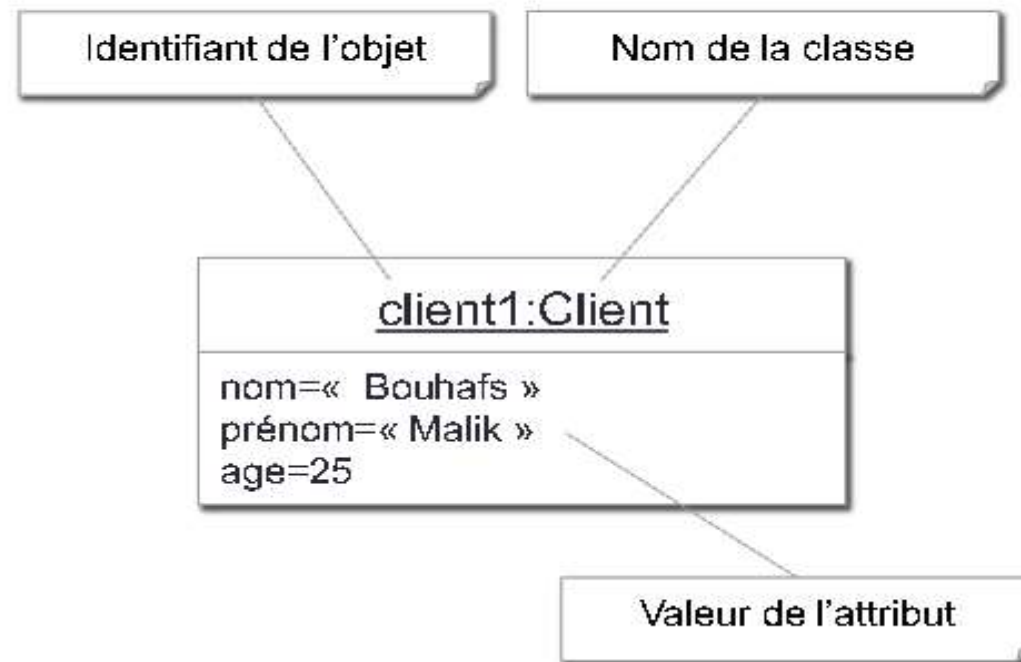
Correspondance UML / JAVA



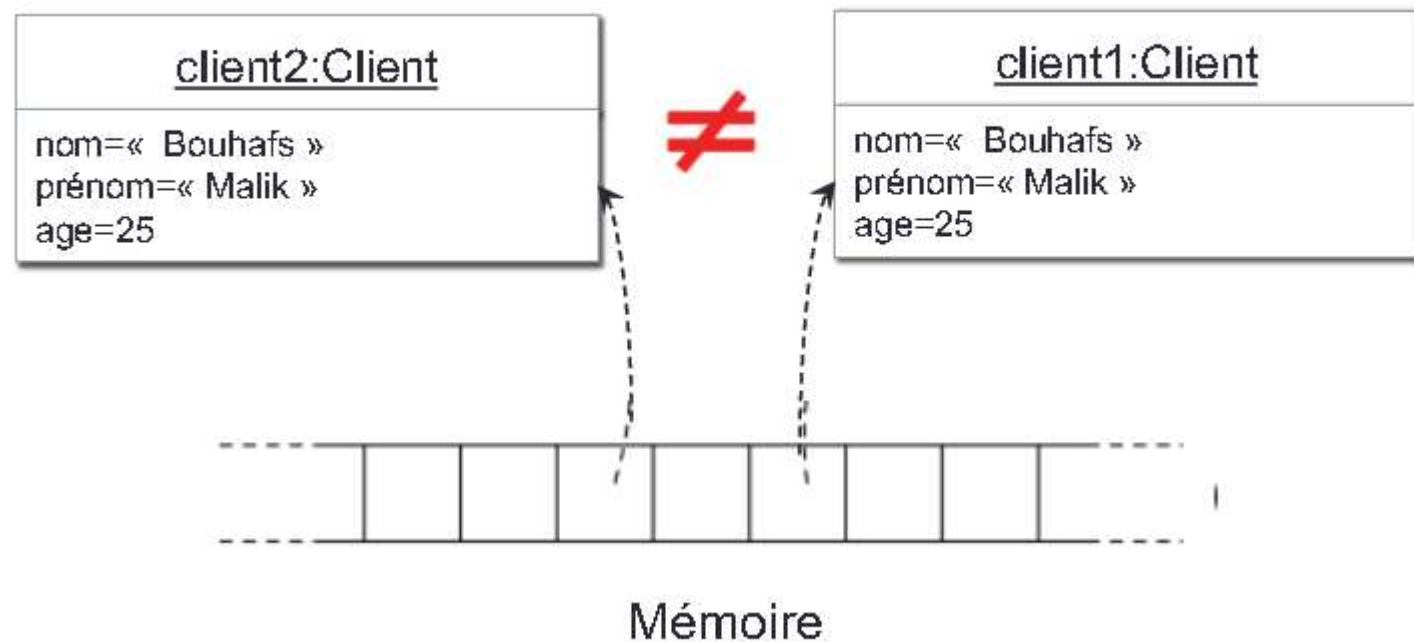
```
class Client{  
  
    String nom;  
    String Prenom;  
    int age;  
  
    boolean retirerArgent(int somme)  
        {return true;}  
  
    void consulterSolde()  
        {System.out.print(solde);}  
  
}
```

Objet

Objet = instance de classe



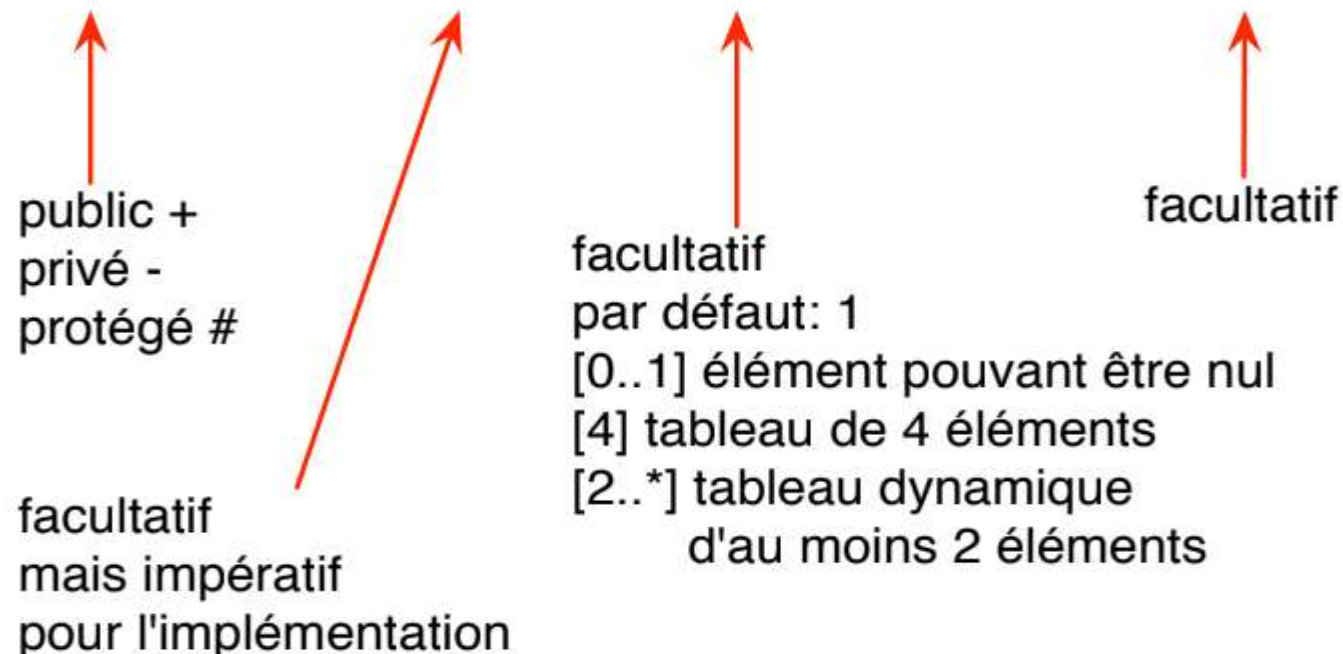
Exemple



Attributs et Opérations

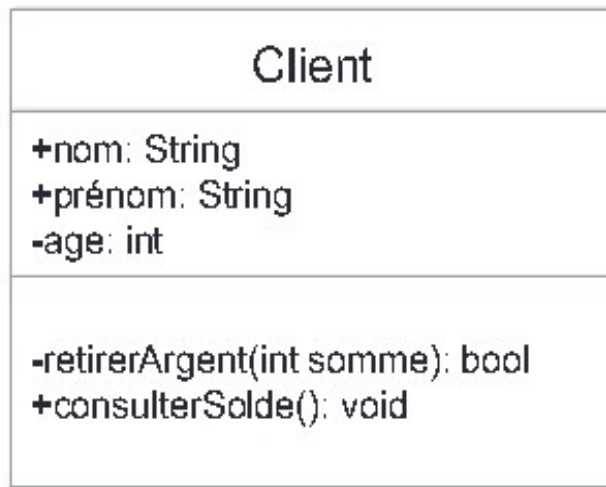
➤ Représentation d'un attribut

visibilité nom : type [multiplicité] = valeur_initiale



Visibilité attributs et méthodes

➤ Représentation UML



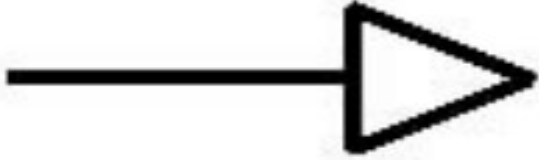
```
class Client{
```

```
    public String nom;  
    public String Prenom;  
    private int age;
```

```
    private boolean retirerArgent(int somme)  
        {return true;}
```

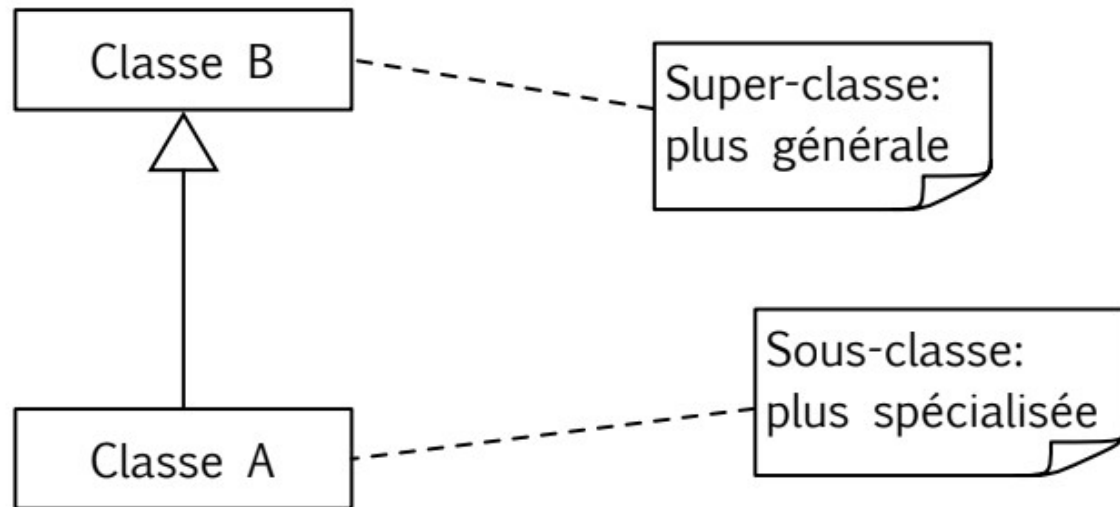
```
    public void consulterSolde()  
        {System.out.print(solde);}  
}
```

Relation entre classe

1.  **Simple**
2.  **Généralisation**
3.  **Agrégation**
4.  **Composition**

2.Héritage

- L'héritage une relation de spécialisation/généralisation.
- Les éléments spécialisés héritent de la structure et du comportement des éléments plus généraux (attributs et opérations)

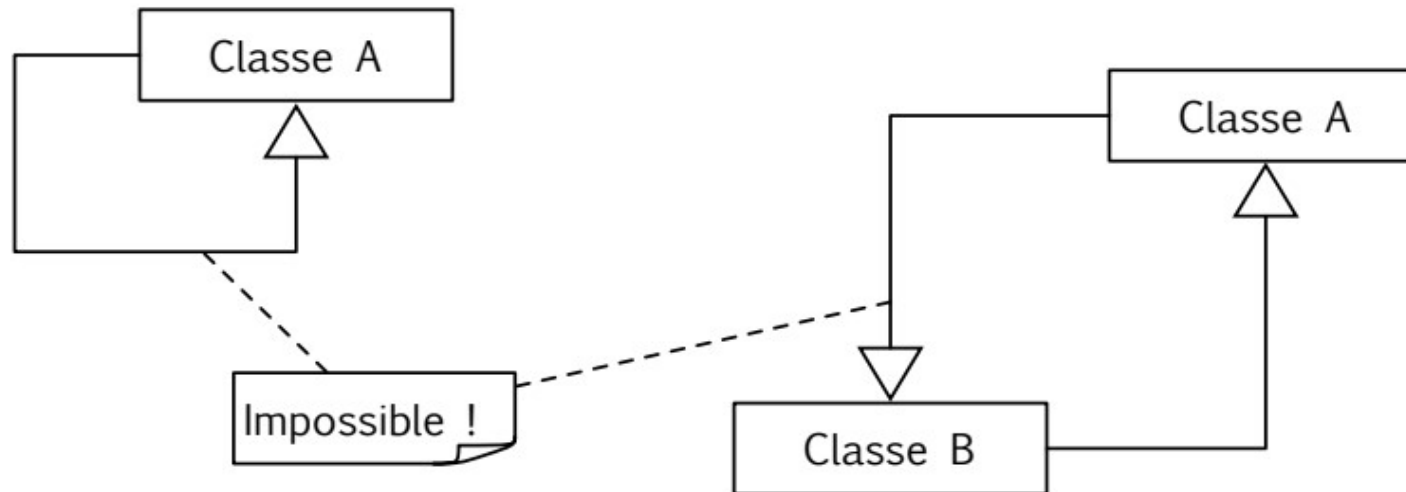


2.Héritage

➤ Pour que ça fonctionne :

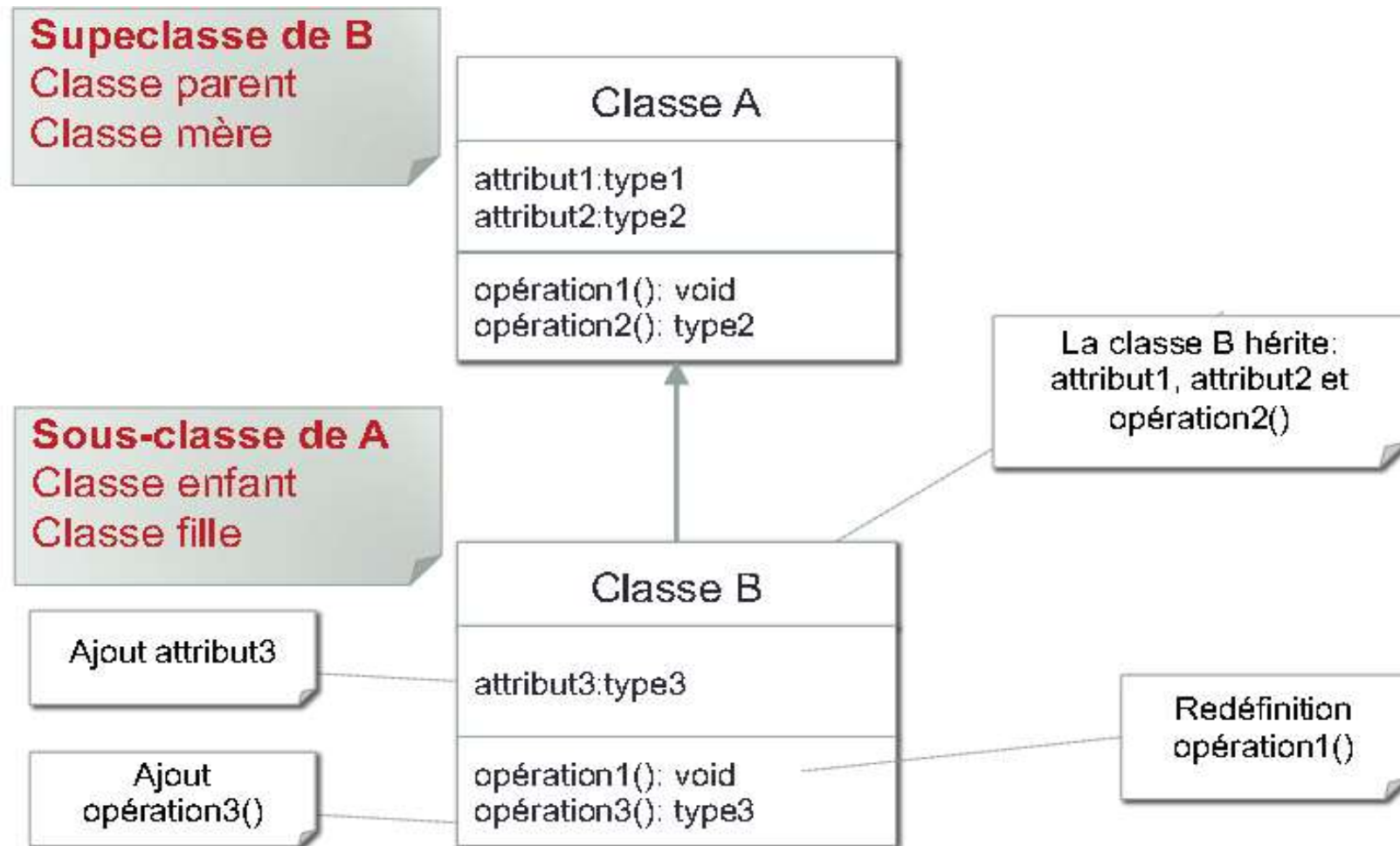
- **Principe de substitution** : toutes les propriétés de la classe parent doivent être valables pour les classes enfant
- **principe du « A est un B » ou « A est une sorte de B »** : toutes les instances de la sous-classe sont aussi instances de la super-classe. **Par exemple**, toute opération acceptant un objet d'une classe Animal doit accepter tout objet de la classe Chat (l'inverse n'est pas toujours vrai).

➤ **Relation non-réflexive, non-symétrique !**

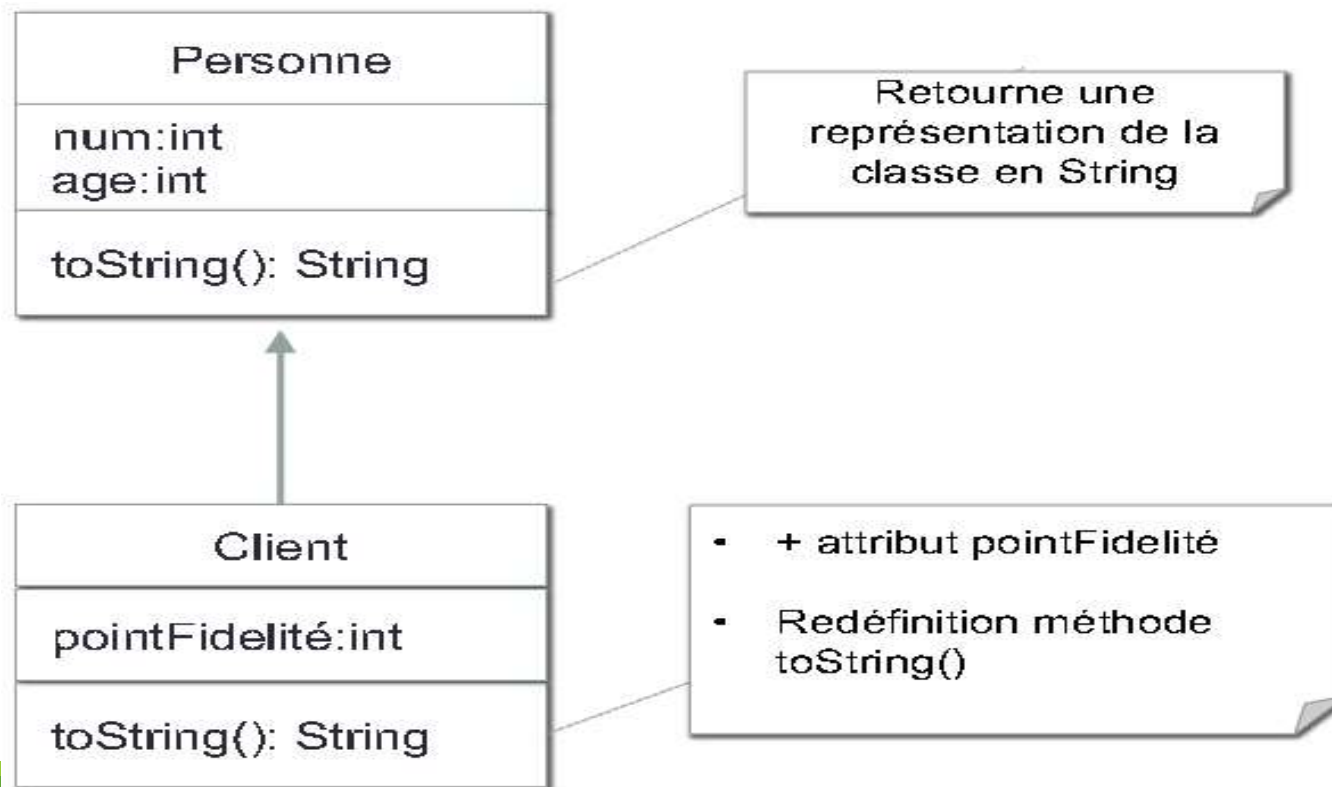


2.Héritage

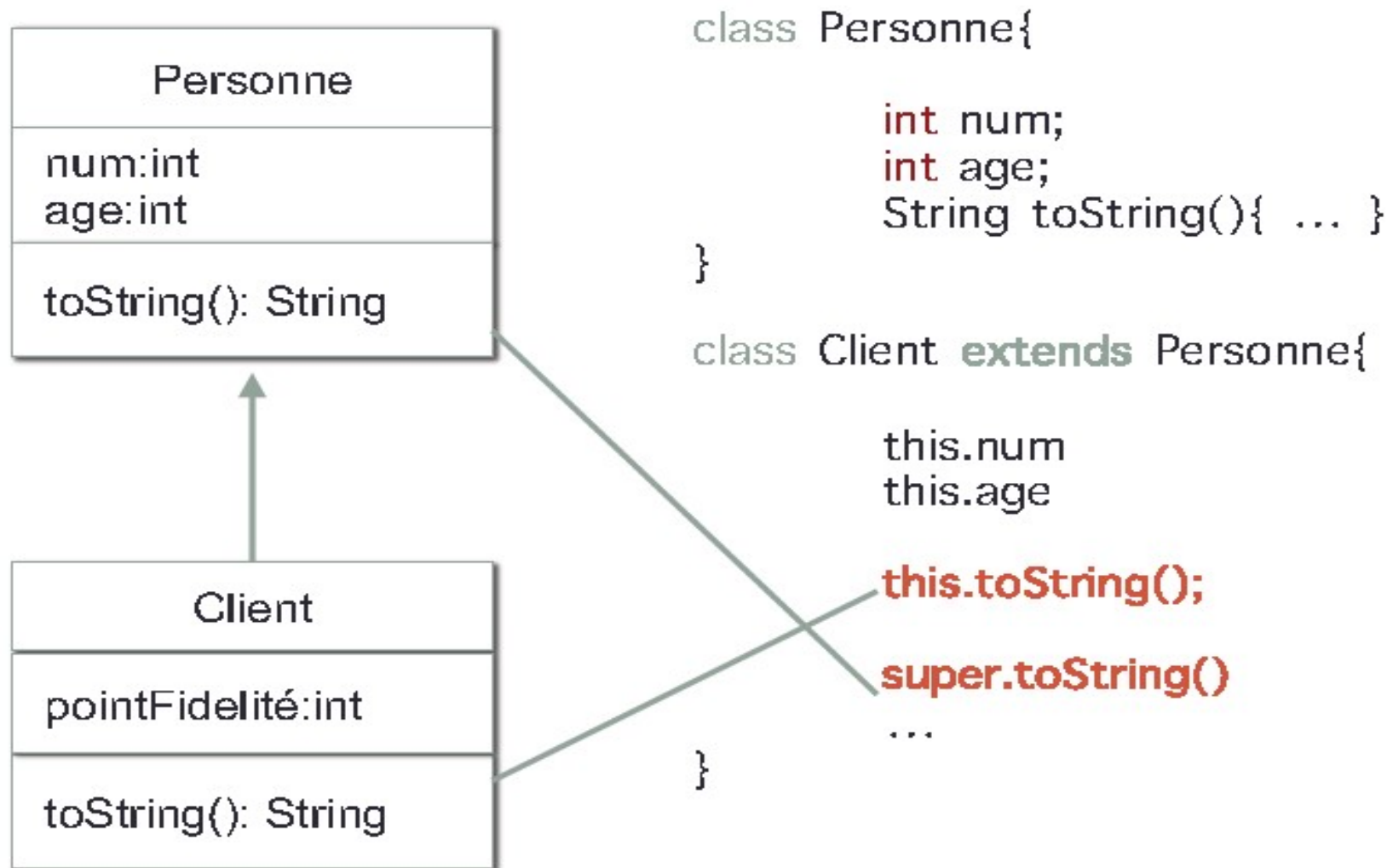
Notation UML



Exemple

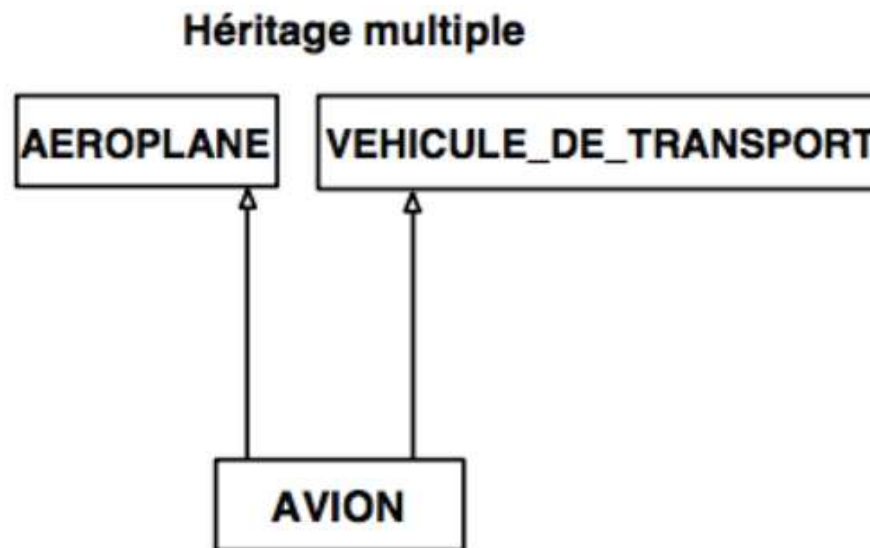


Correspondance UML / JAVA



Héritage multiple

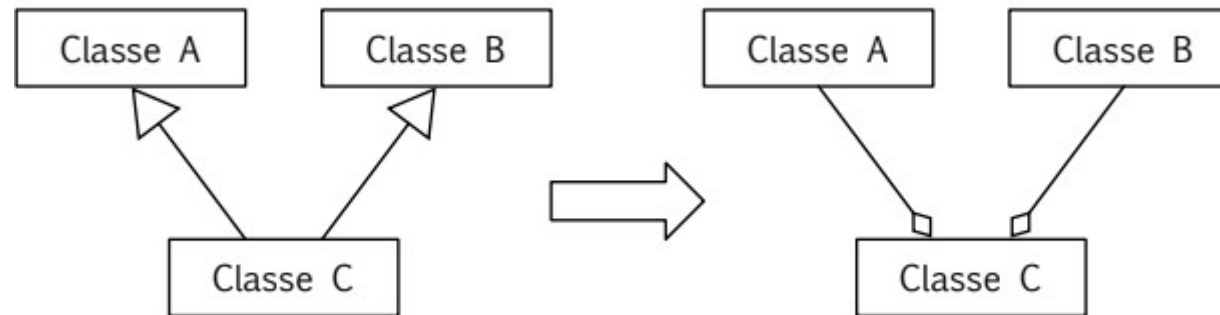
Une classe peut avoir plusieurs classes parents. On parle alors **d'héritage multiple**.



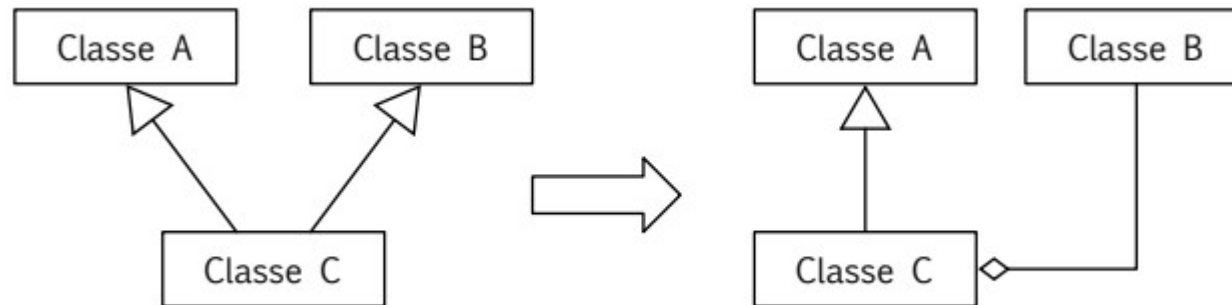
- Incompatible avec **java**
- **langage C++** est un des langages objet permettant son implantation effective.

Comment éviter l'héritage Multiple ?

➤ **Première solution** : déléguer

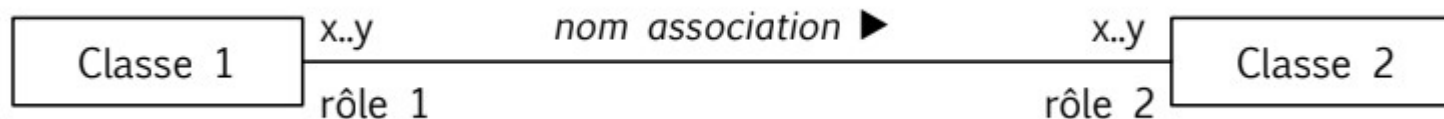


➤ **Deuxième solution**: hériter de la classe la plus importante et déléguer les autres.

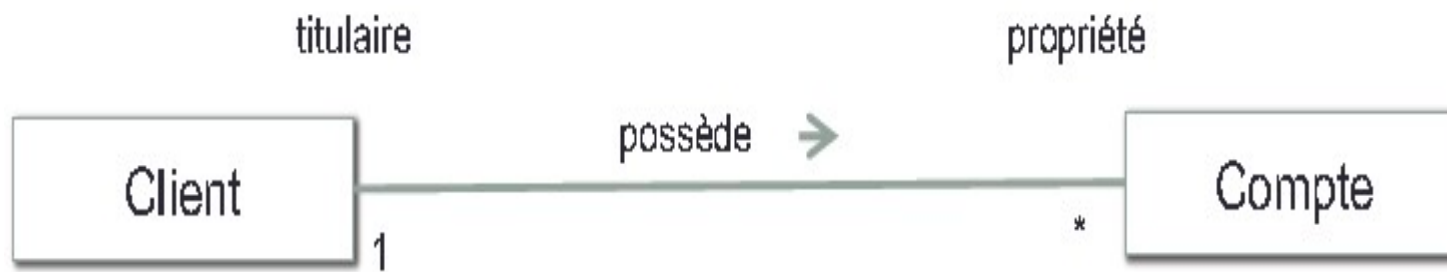


1.Association

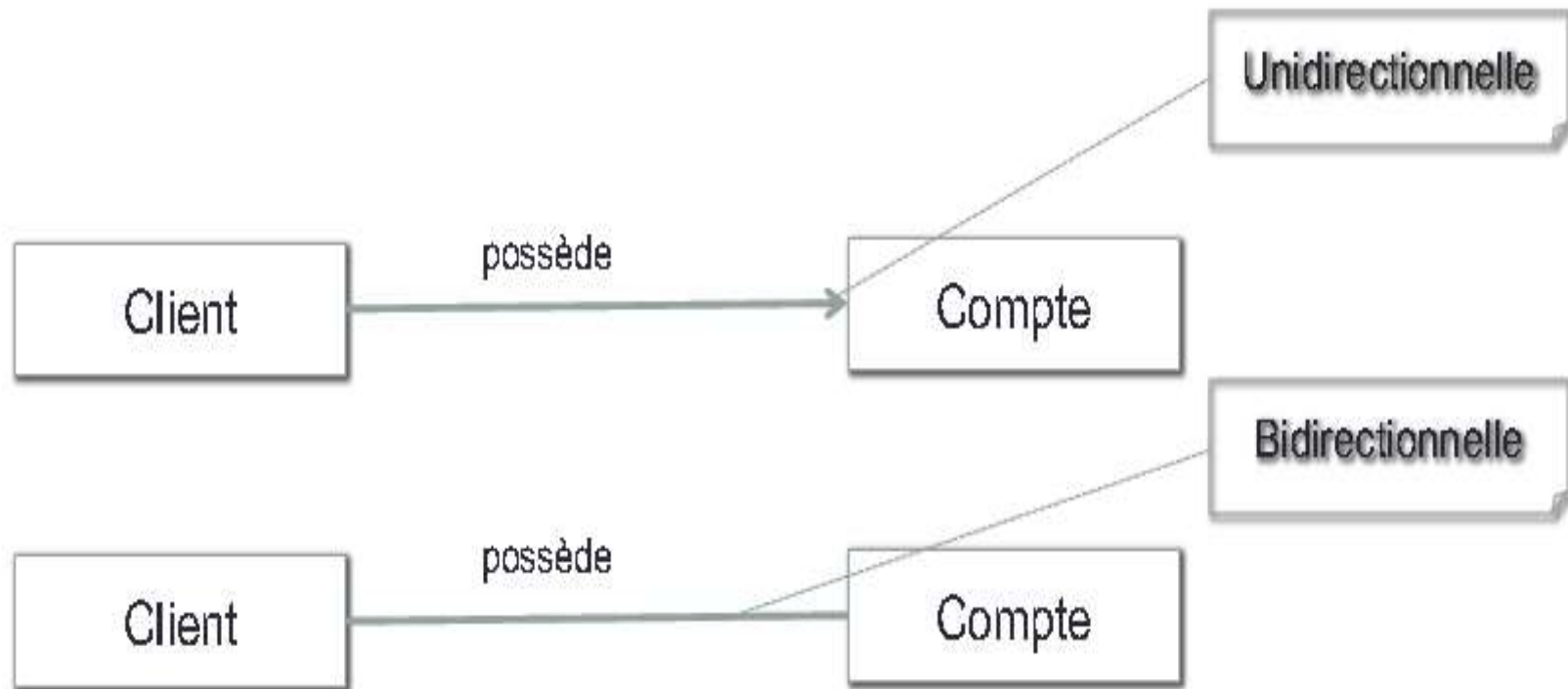
- Connexion sémantique bidirectionnelle entre classes
- Représentation des associations :
 - **Nom** : forme verbale,
 - **Rôles** : forme nominale, décrit une extrémité de l'association
 - **Navigabilité**: sens de lecture
 - **Multiplicité** : 1, 0..1, 0..*, 1..*, n..m



Exemple

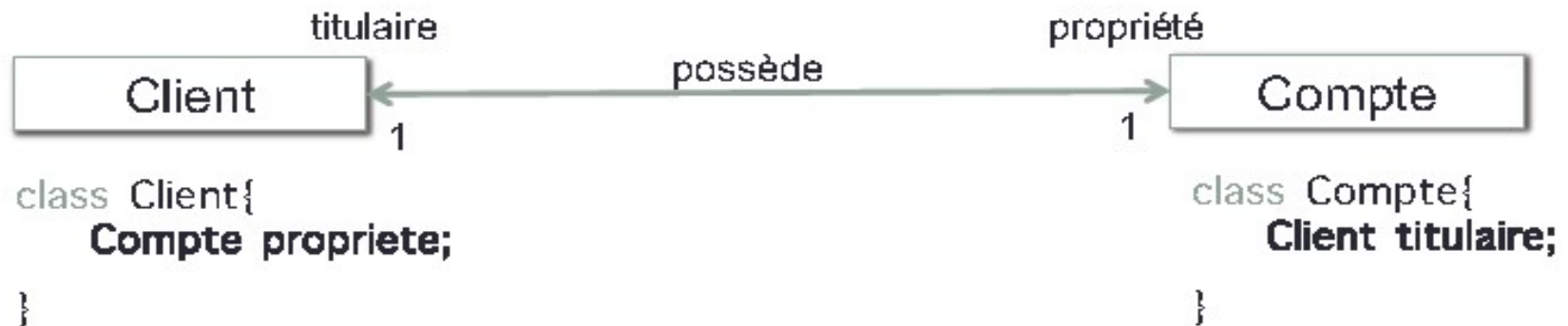


Navigabilité d'une association

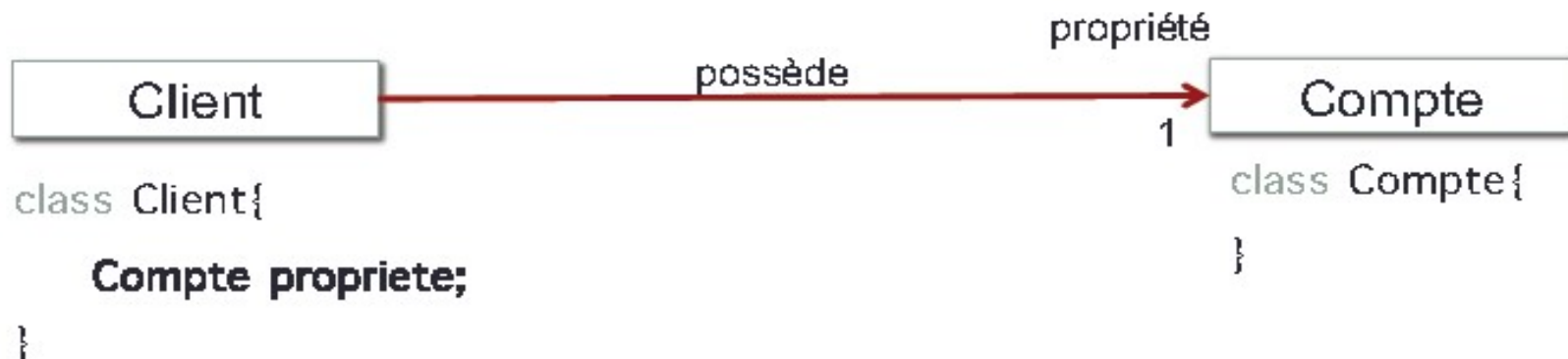


1.Association

■ Association bidirectionnelle

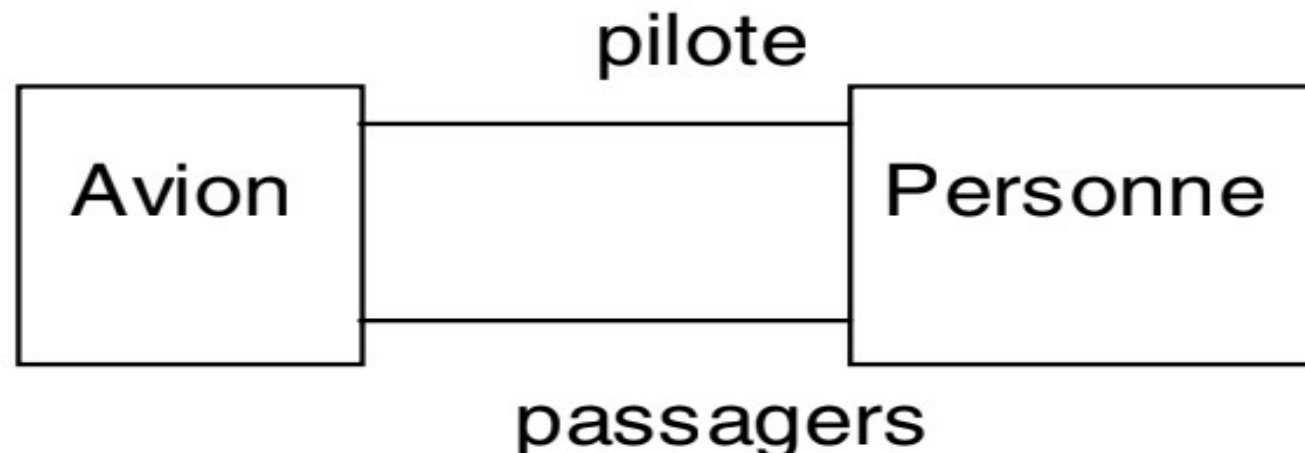
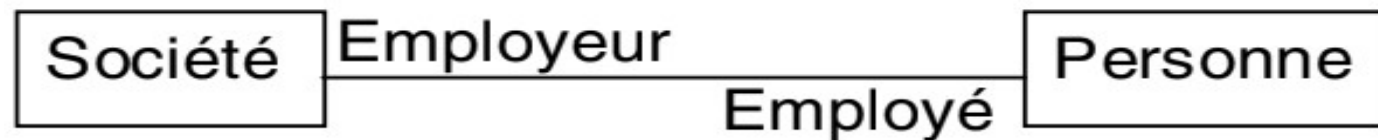


■ Association unidirectionnelle



Nom d'extrémité d'association: Rôle

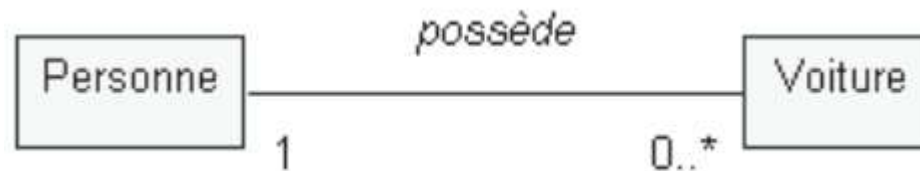
Chaque **extrémité d'association** peut être nommée



Multiplicité des associations

- La multiplicité spécifie le nombre d'instances d'une classe pouvant être liées à une seule instance d'une classe associée. **Elle contraint le nombre d'objets liés.**

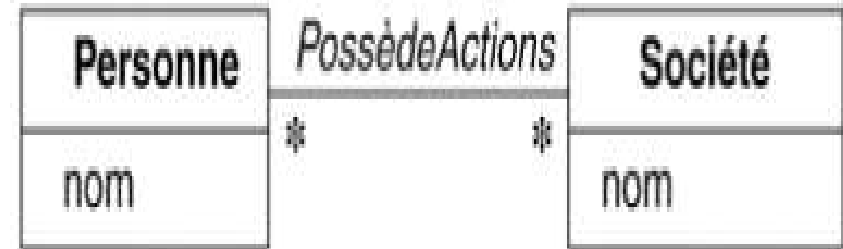
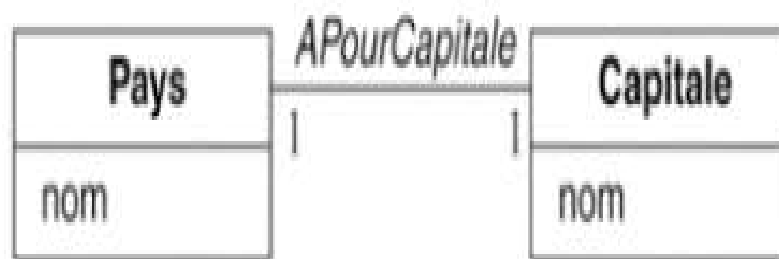
- **Exemple** : une personne peut posséder plusieurs voitures (entre zéro et un nombre quelconque) ; une voiture est possédée par une seule personne.



Multiplicité des associations

1	Un et un seul
0..1	Zéro ou un
N	N (entier naturel)
M .. N	De M à N (entiers naturels)
*	De zéro à plusieurs
0 .. *	De zéro à plusieurs
1 .. *	D'un à plusieurs

Exemple de multiplicité

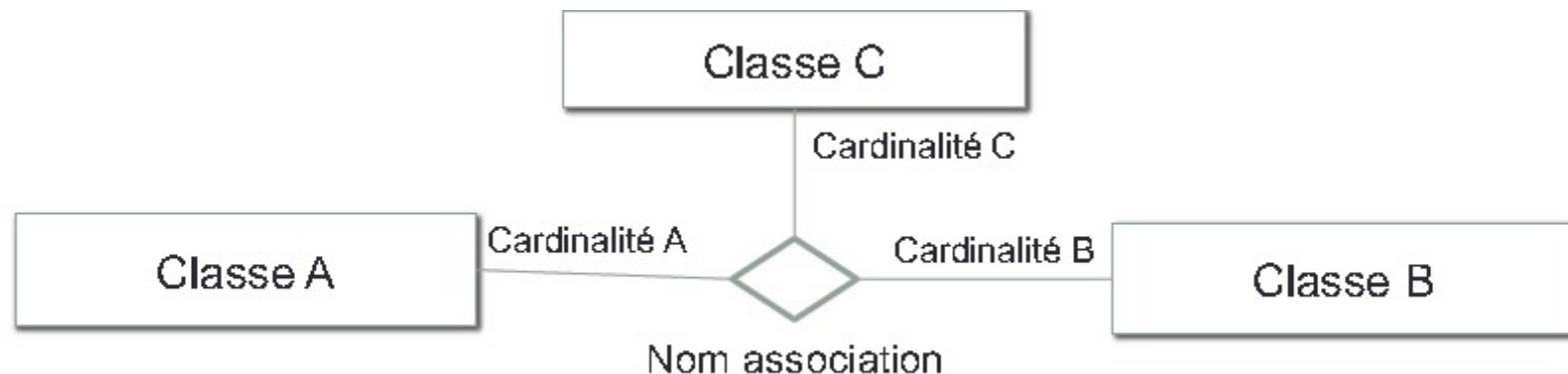


Association n-aire

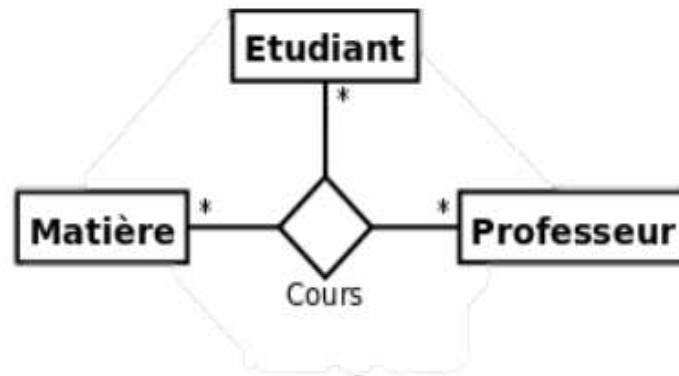
- **En général**, les associations sont binaires
- **N-aires** : au moins trois instances impliquées

!!A utiliser que lorsqu'aucune autre solution n'est possible !

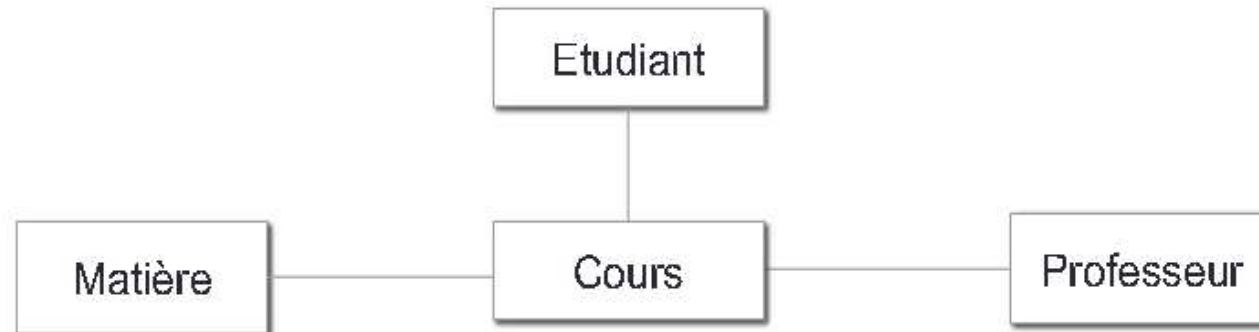
Notation UML



Exemple

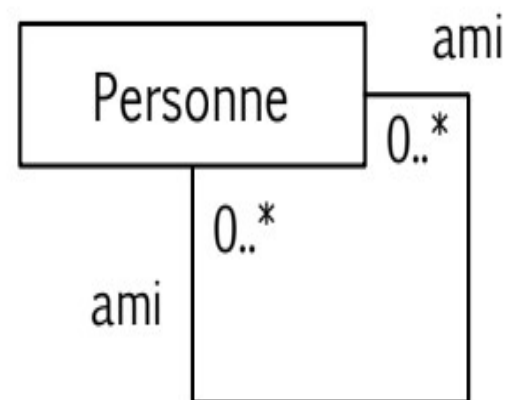
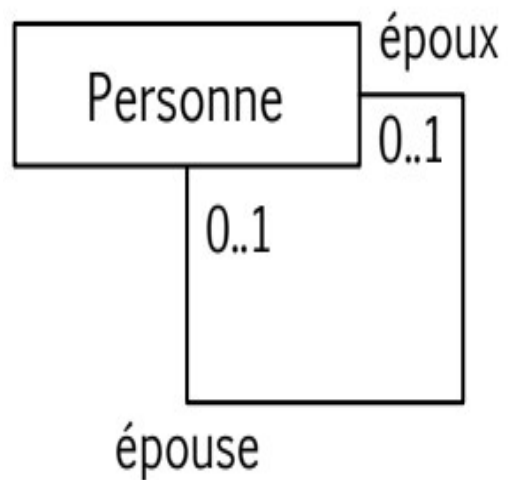


Remplacée par:



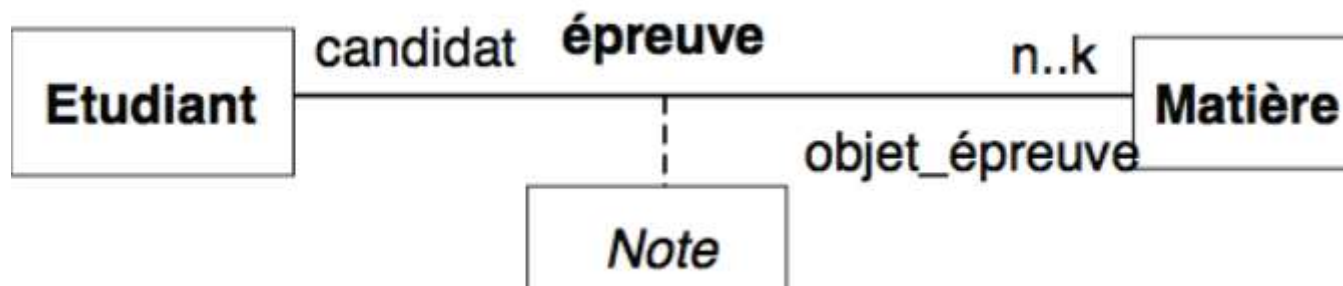
Association réflexive

- Liaison entre des objet de la même classe



Association attribuée

- Inclut une **classe association**
- Informations (attributs, méthodes) spécifiques à l'association.
- Exemple :



3. Agrégation

- Une agrégation est un cas particulier d'association non symétrique exprimant une relation de contenance d'un élément dans un ensemble
- Les agrégations n'ont pas besoin d'être nommées : implicitement elles signifient « contient », « est composé de ».
- On représente l'agrégation par l'ajout d'un losange vide du côté de l'agrégat (l'ensemble).
- **Notation UML**



Exemple



Correspondance JAVA

```
class Polygone{  
    ArrayList<Point> points = new ArrayList<Point>()  
}
```

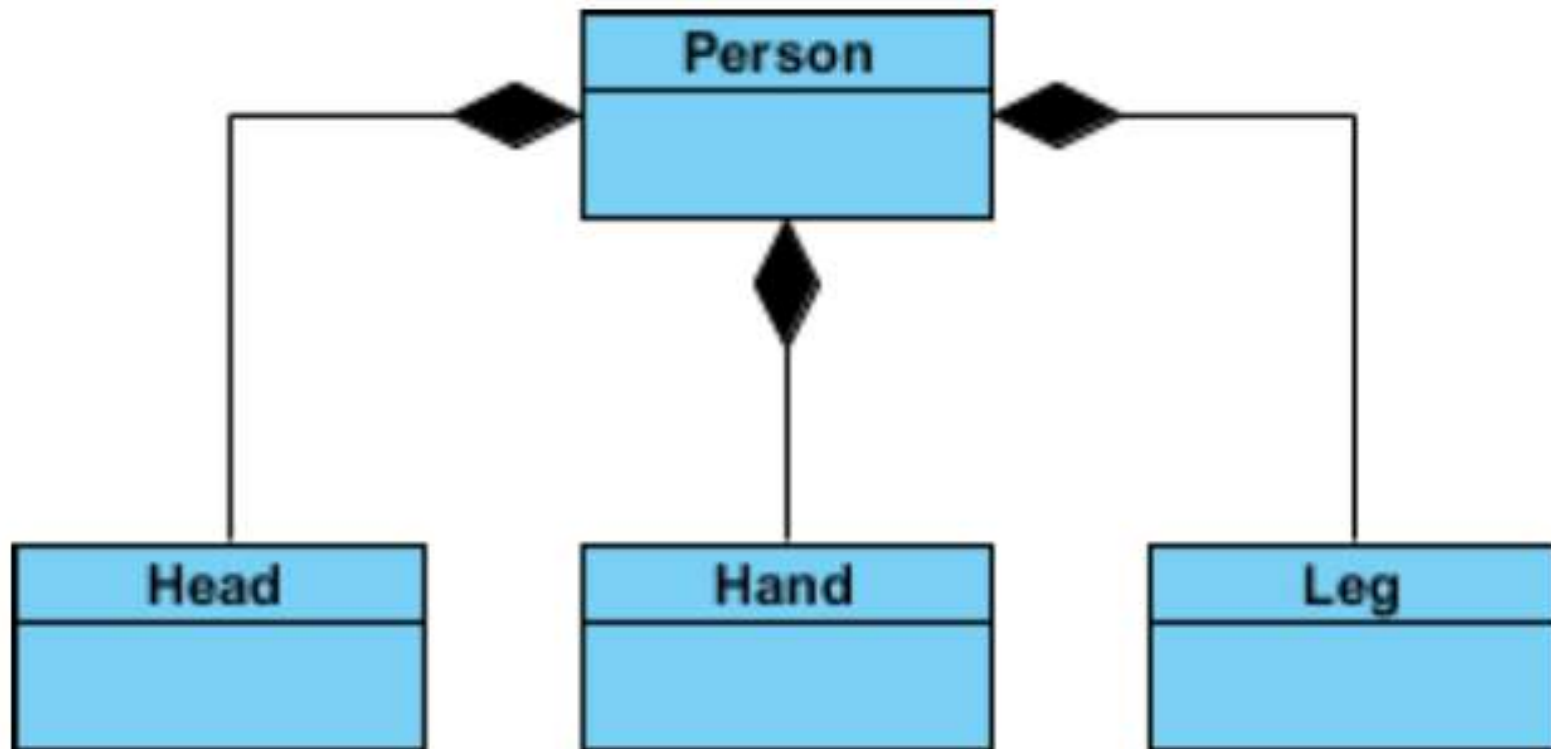
```
class Point{  
}
```

Composition

- Une composition est une agrégation plus forte impliquant que :
 - un élément ne peut appartenir qu'à un seul agrégat composite (agrégation non partagée) ;
 - la destruction de l'agrégat composite (l'ensemble) entraîne la destruction de tous ses éléments (les parties)
- **Notation UML: losange plein**

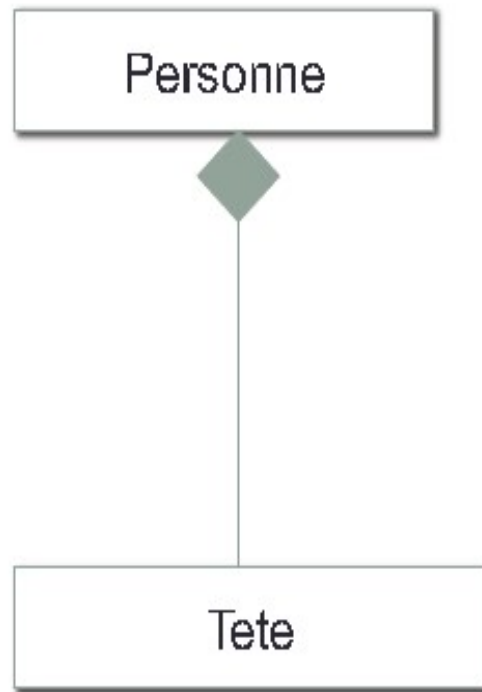


Example



Exemple

➤Correspondance UML/JAVA



```
class Personne{
    Tete tete;
    public Personne(){
        tete =new Tete();
    }
}

class Tete{
}
```

Agrégation vs. composition

➤ Quand mettre une composition plutôt qu'une agrégation ?

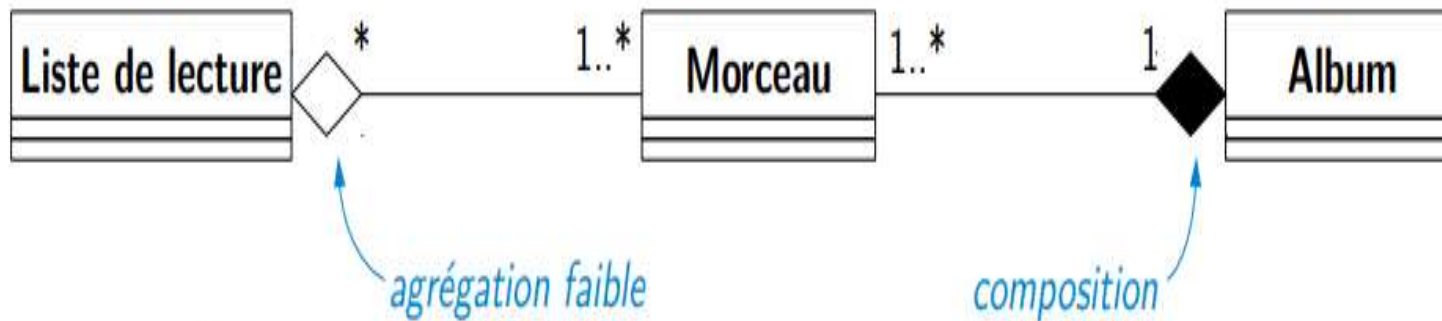
❑ Est-ce que la destruction de l'objet composite (du tout) implique nécessairement la destruction des objets composants (les parties) ? C'est le cas si les composants n'ont pas d'autonomie vis-à-vis des composites.

❑ Lorsque l'on copie le composite, doit-on aussi copier les composants, ou est-ce qu'on peut les «réutiliser», auquel cas un composant peut faire partie de plusieurs composites ?

➤ Si on répond par l'affirmative à ces deux questions, on doit utiliser une **composition**.

Exemple

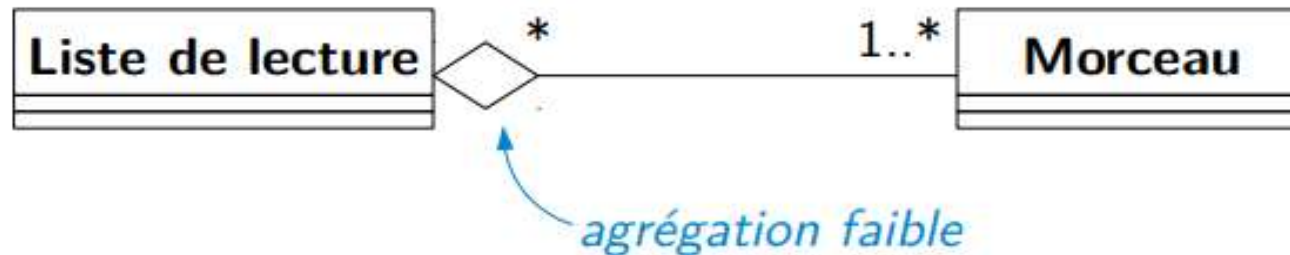
Lecteur de contenu audio permettant de créer des listes de lecture



Exemple

- Une liste de lecture est composée d'un ensemble de morceaux
- Un morceau peut appartenir à plusieurs listes de lecture

Supprimer la liste ne supprime pas les morceaux



Exemple

- Un morceau n'appartient qu'à un album
- La suppression de l'album entraîne la suppression de tous ses morceaux



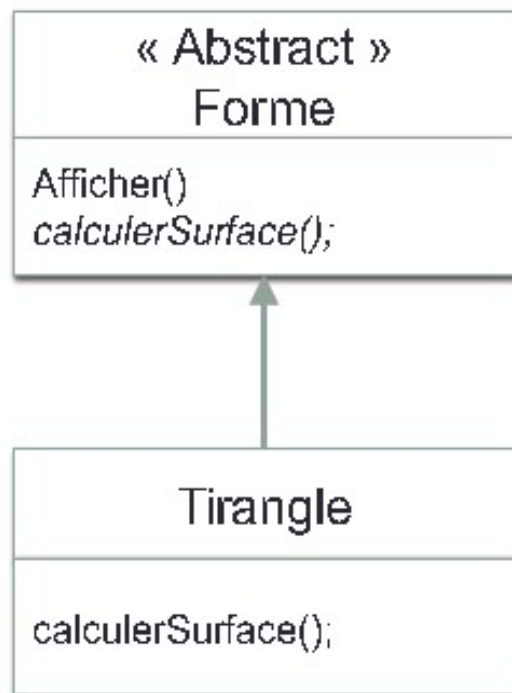
Diagramme de classes

Concepts avancés

Classes abstraites

- Une méthode est dite **abstraite** lorsqu'on connaît son entête (signature) mais pas la manière dont elle peut être réalisée.
 - Il appartient aux classes enfant de définir les méthodes abstraites.
- Une classe est dite **abstraite** lorsqu'elle définit au moins une méthode abstraite ou lorsqu'une classe parent contient une méthode abstraite non encore réalisée.

Exemple



```
abstract class Forme {  
    void afficher(){System.out.print();}  
    abstract int calculerSurface();  
}  
  
class Triangle extends Forme {  
    @Override  
    int calculerSurface() {  
        // TODO Auto-generated method stub  
        return 0;  
    }  
}  
....  
Forme f = new Triangle();
```

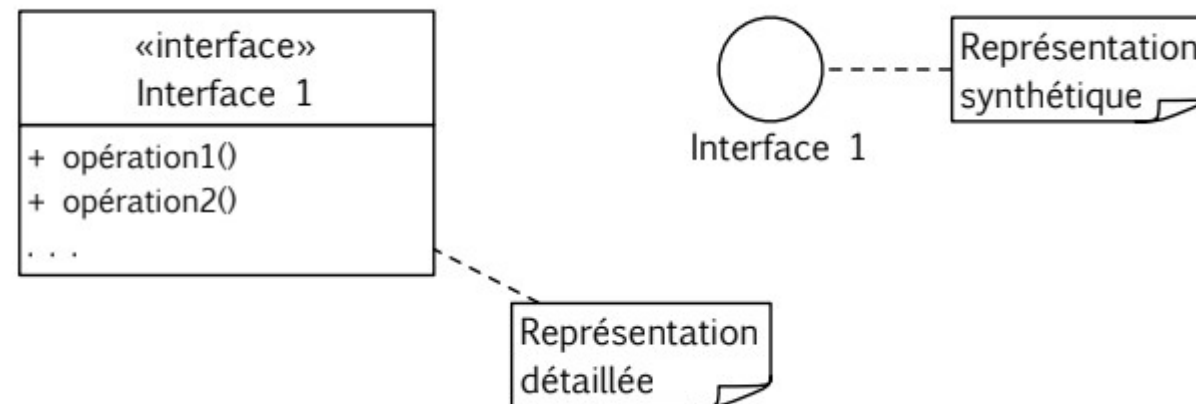
Les interfaces

- Pas une classe
- Liste de méthodes abstraites
- Pas d'attributs (sauf constante « final static.. »)
- Définie un service et ne peut pas servir à créer des objets
- Implémenté par au moins une classe concrète.

Les interfaces

- C'est un contrat
 - les classes liées s'engagent à respecter le contrat
 - elles doivent mettre en œuvre les opérations de l'interface

En UML :



Relation avec une interface

➤ Réalisation

Implémentation de l'interface

- Définitions de toutes les méthodes abstraites

Une classe peut réaliser plusieurs interfaces

Une interface peut être réalisée par plusieurs classes

• Notation UML

- Héritage avec une ligne en pointillée
- Stéréotype « realize »



➤ Utilisation / dépendance

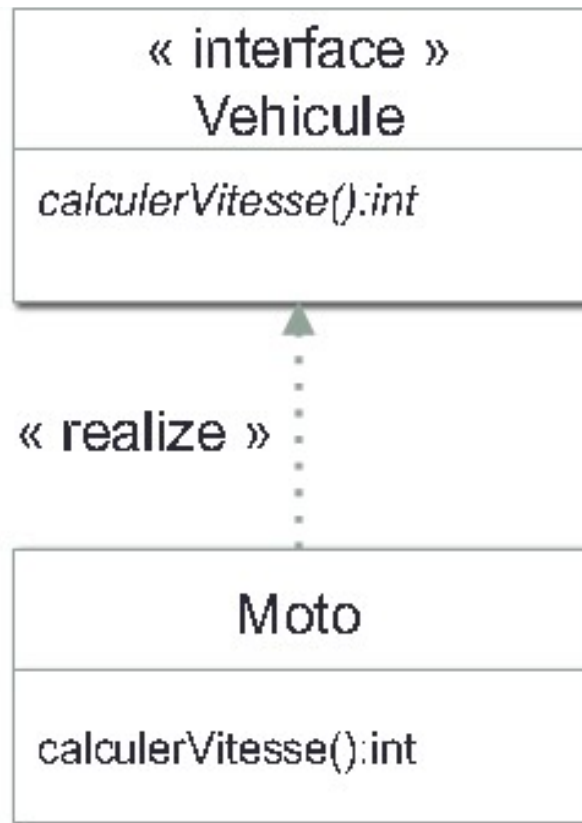
- Utilisation des fonctionnalités offertes par l'interface

Notation UML

- dépendance
- Stéréotype «use »



Réalisation d'interface



```
interface Vehicule{
```

```
int calculerVitesse();
```

```
}
```

```
class Moto implements Vehicule{
```

```
@Override
```

```
int calculerVitesse(){
```

```
// TODO Auto-generated method stub
```

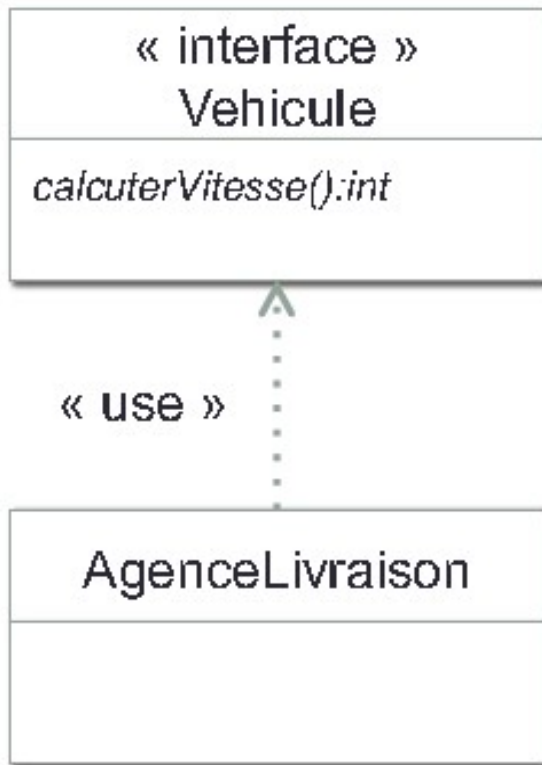
```
....
```

```
return vitesse;
```

```
}
```

```
}
```

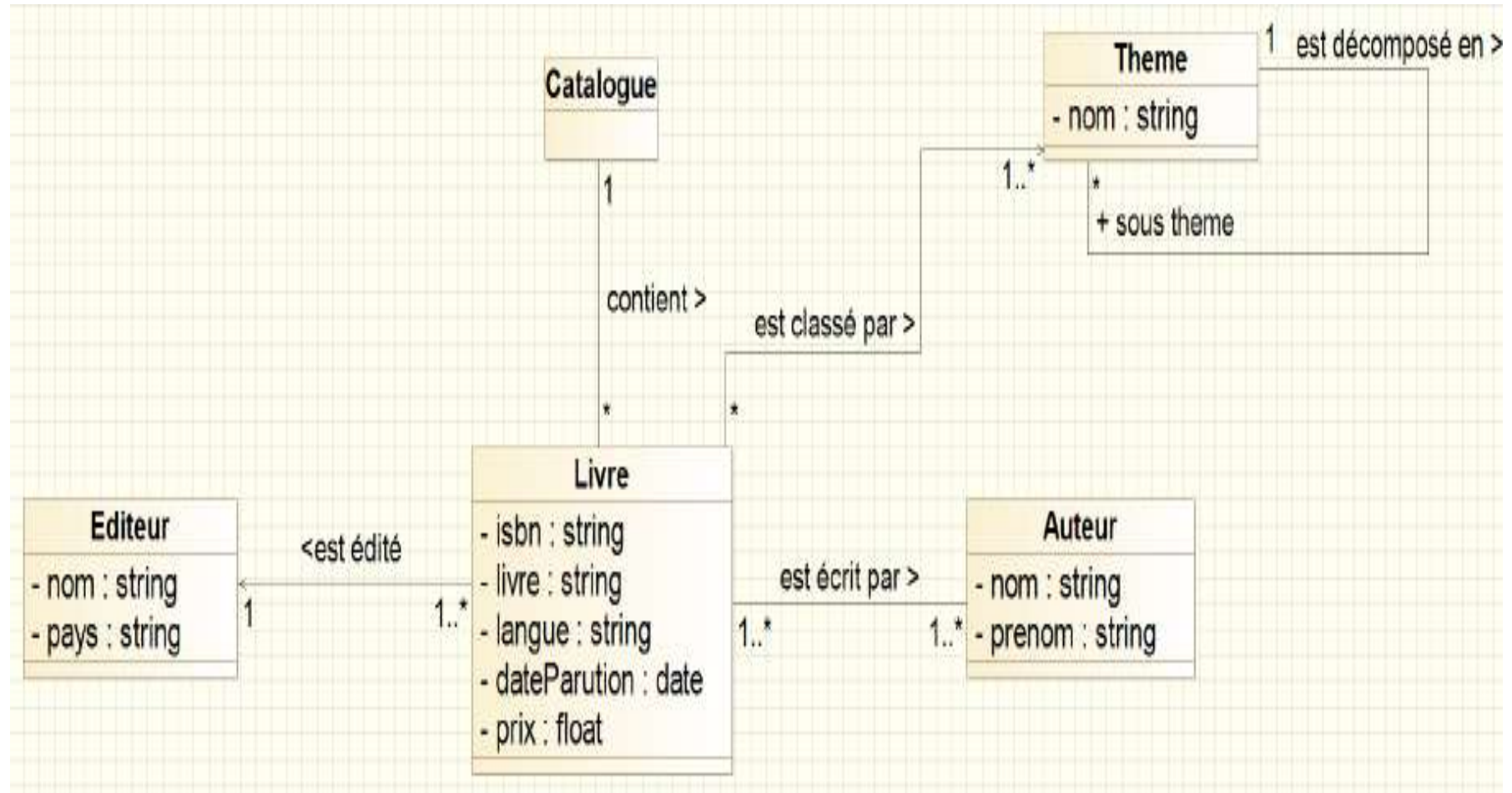
Dépendance d'interface



```
interface Vehicule{
    int calculerVitesse();
}

class AgenceLivraison{
    void livraison (Vehicule v) {
        ...
        v.calculerVitesse();
        ...
    }
}
```

Exemple: Diagramme de classe



Méthodologie

1. Trouvez les classes du domaine étudié (les noms)
2. Trouvez les associations entre classes (les verbes qui relient les noms)
3. Affinez le diagramme en éliminant les classes et les associations redondantes, non pertinentes
4. Une fois que les classes sont bien établies cherchez les relations d'héritage et d'agrégation
5. Ajoutez les attributs de chaque classe
6. Vérifiez que vous pouvez réaliser les cas d'utilisation par parcours du diagramme de classe